

Báo cáo AI Thực Tập Cơ Sở

morex3

Ngày 19 tháng 5 năm 2026

Mục lục

1	Item-Item Collaborative Filtering	5
1.1	1. Item similarity	5
1.2	2. Item-item collaborative	6
1.3	3. Baseline của Collaborative	6
1.4	4. Kết hợp một số ý tưởng	7
1.5	5. Cách tính weight	7
1.6	6. Sự lựa chọn item hàng xóm	7
1.7	7. Tổng kết thuật toán Item-Item Collaborative	7
1.8	8. Độ phức tạp	8
1.9	9. Điểm mạnh	8
1.10	10. Điểm yếu	8
2	Ví dụ Cụ Thể Item-Item CF	9
2.1	1. Dữ liệu ví dụ	9
2.2	2. Chọn neighborhood	9
2.3	3. Tính average rating của từng item	9
2.4	4. Tính weight giữa item B và A	10
2.5	5. Ý nghĩa của weight cao	10
2.6	6. Tính weight giữa B và C	10
2.7	7. Predict rating cho U1 với item B	11
2.8	8. Rating dự đoán cuối cùng	11
2.9	9. Giải thích trực quan	11
2.10	10. Ý nghĩa của mean-centering	11
2.11	11. Tổng kết thuật toán Item-Item CF	11
3	Content-based Filtering	12
3.1	1. Giới thiệu	12
3.2	2. Thuật ngữ	12
3.3	3. Workflow	13
3.4	4. Item Content	13
3.5	5. Cách biểu diễn vector các mặt hàng	14
3.6	6. Mô hình không gian vector (Vector Space Model)	14
3.7	7. Bag of word	14
3.8	8. Count frequencies	15
3.9	9. Weigh frequencies	15
3.10	10. Term Frequencies	15
3.11	11. Inverse document frequency	15
3.12	12. Example of after tf-idf	16

3.13	13. User Profile + Matching	16
3.14	14. Relevant feedback	17
3.15	15. Ví dụ: Cập nhật Profile người dùng theo sách	17
4	Hệ thống Hybrid Recommender đơn giản	21
4.1	1. Tại sao cần Hybrid Recommender?	21
4.2	2. Công thức cộng trọng số đơn giản	22
4.3	3. Lưu ý về chuẩn hóa điểm số	23
4.4	4. Ví dụ cụ thể	23
4.5	5. Quy trình thuật toán Hybrid đơn giản	24
4.6	6. Cách chọn hệ số λ	25
4.7	7. Ưu điểm và hạn chế	25
4.8	8. Tổng kết	26

Danh sách hình vẽ

1.1	So sánh độ ổn định giữa Item similarity và User similarity	6
3.1	Giới thiệu cơ bản về Content-based Filtering	12
3.2	Sơ đồ Workflow của hệ thống Content-based Filtering	13
3.3	Các dạng mô tả của Item Content	14
3.4	Mô hình Bag of word	15
3.5	Mô hình Count frequencies	15
3.6	Minh họa Inverse document frequency (IDF)	16
3.7	Ví dụ minh họa sau khi áp dụng tính toán TF-IDF	16
3.8	Công thức Rocchio trong User Profile Matching	16
3.9	Biểu diễn vector sở thích ban đầu u_0 trên không gian 2 chiều	18
3.10	Sự dịch chuyển của vector sở thích u sau khi cập nhật bằng công thức Rocchio	19
3.11	So sánh độ tương đồng cosine giữa profile người dùng u với Sách X và Sách Y	20

Danh sách bảng

2.1	Bảng rating dữ liệu mẫu	9
2.2	Phản ứng của người dùng đối với Item A và B	10
3.1	Các phương pháp xác định vector cơ sở u_0 tùy ngữ cảnh hệ thống	17

Chương 1

Item-Item Collaborative Filtering

1.1 1. Item similarity

1.1. Câu hỏi đặt ra: Làm sao chúng ta có thể tính **độ tương đồng** của các item mà **không cần nội dung, mô tả...**

- Độ tương đồng có thể được **định lượng** bằng cách so sánh tập các người dùng thích chúng → cụ thể là rating của họ
- Hai sản phẩm là tương đồng nếu như nó nhận rating tương tự giữa các users

VD1: nếu A thích item 1, Item 2. B ghét item 1, item 2 → item 1 và item 2 tương tự nhau.

Vì sao người thích người ghét lại tương tự? “**Cùng gây ra một loại phản ứng giống nhau từ phía người dùng**”

Ví dụ: Sách trinh thám cực kỳ “hack não”

- **Người mê trinh thám** sẽ đánh giá:
 - Sách A: ★★★★★
 - Sách B: ★★★★★
- **Người không thích trinh thám / thích đọc nhẹ nhàng** sẽ đánh giá:
 - Sách A: ★
 - Sách B: ★

⇒ Dù mức đánh giá khác nhau hoàn toàn (5 sao vs 1 sao), nhưng cả hai nhóm người dùng đều có **xu hướng giống nhau**: Thích hoặc ghét **cùng một loại sách**.

Đây là lý do hệ thống gợi ý trên web bán sách có thể nhận ra:

- Hai cuốn sách A và B “liên quan” với nhau.
- Người từng thích A có khả năng thích B.

1.2. Item similarity **stable** hơn user similarity

Tại sao?

- Do số lượng người dùng nhiều hơn item $\rightarrow$ trung bình một item có **nhiều rating hơn** trung bình 1 người dùng.
- Nội dung của item thì lại ít thay đổi nếu có rating mới $\rightarrow$ stable.
- Khắc phục được các nhược điểm của U-U CF (sở thích dynamic, mở rộng, thừa thớt dữ liệu).

	Item 1 (Phim A)	Item 2 (Phim B)	Số lượng ratings của User
User 1	5		1
User 2	4	2	2
User 3		5	1
User 4	2		1
User 5		4	1
Số lượng ratings của Item	3	3	

Hình 1.1: So sánh độ ổn định giữa Item similarity và User similarity

1.2 2. Item-item collaborative

Ý tưởng:

1. Tính độ tương đồng của target item với tất các item khác
2. Tạo neighborhood
3. "Dự đoán đánh giá (rating) bằng cách tính **trung bình có trọng số** các điểm đánh giá mà **người dùng mục tiêu** đã cho đối với các **mục tương tự (láng giềng)** của món đồ đang xét."

1.3 3. Baseline của Collaborative

Công thức Baseline tính điểm dự đoán:

$$s(u, i) = \frac{\sum_{j \in I_u} r_{uj}}{|I_u|}$$

Trong đó tập các item mà user u đã đánh giá được định nghĩa:

$$I_u = \{i \in I \mid r_{ui} \in R\}$$

1.4 4. Kết hợp một số ý tưởng

- Có phải tất cả item có đóng góp công bằng không? → Không
- Giải quyết → Lựa chọn Neighborhood là những item có đóng góp cao nhất để predict
- Chuẩn hóa mean-center:

$$s(u, i) = \bar{r}_i + \frac{\sum_{j \in N(i)} w_{ij}(r_{uj} - \bar{r}_j)}{\sum_{j \in N(i)} |w_{ij}|}$$

Tại sao lại lấy rating của người dùng u đánh giá đối với item j trừ đi rating trung bình trên hệ thống của item j ? → Nhờ điều này mà chúng ta có thể biết so với mặt bằng chung thì người dùng u sẽ là người dùng khó tính hơn hay dễ tính đối với item này và phần w_{ij} chính là đóng góp của item. Mẫu số để chuẩn hóa về thang đo ban đầu.

1.5 5. Cách tính weight

Hệ số Pearson Correlation → Giống với mean-center cosin:

$$w_{ij} = \frac{\sum_{u \in U_i \cap U_j} (r_{ui} - \bar{r}_i)(r_{uj} - \bar{r}_j)}{\sqrt{\sum_{u \in U_i} (r_{ui} - \bar{r}_i)^2} \sqrt{\sum_{u \in U_j} (r_{uj} - \bar{r}_j)^2}}$$

Trong công thức này có thể giải thích cho ví dụ tại sao các item có cùng phản ứng với cùng sản phẩm thì sẽ có độ tương đồng cao.

1.6 6. Sự lựa chọn item hàng xóm

- Để chọn hàng xóm cho target item cho target user thì chúng ta chọn các item mà target user đó đã rate.
- Về số lượng thì có nhiều cách giới hạn và tiêu chuẩn thực tế là 20 neighbors.

1.7 7. Tổng kết thuật toán Item-Item Collaborative

Nhắc lại công thức Weight (Pearson) và Predict (Mean-centered):

$$w_{ij} = \frac{\sum_{u \in U_i \cap U_j} (r_{ui} - \bar{r}_i)(r_{uj} - \bar{r}_j)}{\sqrt{\sum_{u \in U_i} (r_{ui} - \bar{r}_i)^2} \sqrt{\sum_{u \in U_j} (r_{uj} - \bar{r}_j)^2}}$$
$$s(u, i) = \bar{r}_i + \frac{\sum_{j \in N(i)} w_{ij}(r_{uj} - \bar{r}_j)}{\sum_{j \in N(i)} |w_{ij}|}$$

Cuối cùng thì chúng chỉ cần gợi ý cho user các item có rating được dự đoán cao nhất.

1.8 8. Độ phức tạp

Ký hiệu: m là số lượng user, n là số lượng item.

- Computing similarity between two items takes $\mathcal{O}(m)$
- Computing all pairwise item similarities takes $\mathcal{O}(n^2m)$
- Creating a neighborhood of fixed size k takes $\mathcal{O}(n)$
- Predicting rating for a single item takes $\mathcal{O}(k)$
- Selecting the item with the highest predicted rating takes $\mathcal{O}(n)$

Chi phí chủ yếu bị chi phối bởi bước tính toán độ tương đồng (bậc 2 đối với số lượng item).

1.9 9. Điểm mạnh

- Hiệu quả hơn U-U
- Stable hơn

1.10 10. Điểm yếu

- Không tạo ra độ bất ngờ bằng U-U $\rightarrow$ Bản chất vẫn là các item tương tự

Chương 2

Ví dụ Cụ Thể Item-Item CF

Ta sẽ làm một ví dụ **Item-Item Collaborative Filtering** hoàn chỉnh theo đúng các công thức trong hình, với ngữ cảnh **web bán sách**.

2.1 1. Dữ liệu ví dụ

Giả sử web bán sách có 3 cuốn:

- **A** = Sherlock Holmes
- **B** = Ấn mạng trên chuyến tàu tốc hành
- **C** = Doraemon

Ta cần dự đoán xem **User U1** sẽ thích sách **B** bao nhiêu sao.

Bảng 2.1: Bảng rating dữ liệu mẫu

User	A	B	C
U1	5	?	1
U2	5	4	1
U3	4	5	2
U4	1	1	5

Ta muốn predict: $s(U1, B)$

2.2 2. Chọn neighborhood

Theo Item-Item CF: Để predict cho item B của user U1, ta chọn các item mà U1 đã rate: A và C. Tức là:

$$N(B) = \{A, C\}$$

2.3 3. Tính average rating của từng item

- Mean của A: $\bar{r}_A = \frac{5+5+4+1}{4} = 3.75$
- Mean của B: $\bar{r}_B = \frac{4+5+1}{3} = 3.33$

- Mean của C: $\bar{r}_C = \frac{1+1+2+5}{4} = 2.25$

2.4 4. Tính weight giữa item B và A

Các user cùng rate A và B là: U2, U3, U4.

$$w_{ij} = \frac{\sum_{u \in U_i \cap U_j} (r_{ui} - \bar{r}_i)(r_{uj} - \bar{r}_j)}{\sqrt{\sum_{u \in U_i} (r_{ui} - \bar{r}_i)^2} \sqrt{\sum_{u \in U_j} (r_{uj} - \bar{r}_j)^2}}$$

Tử số:

- U2: $(4 - 3.33)(5 - 3.75) = 0.67 \times 1.25 = 0.8375$
- U3: $(5 - 3.33)(4 - 3.75) = 1.67 \times 0.25 = 0.4175$
- U4: $(1 - 3.33)(1 - 3.75) = (-2.33) \times (-2.75) = 6.4075$

Tổng tử số = 7.6625

Mẫu số:

- Phần của B: $(4 - 3.33)^2 + (5 - 3.33)^2 + (1 - 3.33)^2 = 8.6667 \rightarrow \sqrt{8.6667} = 2.944$
- Phần của A: $(5 - 3.75)^2 + (4 - 3.75)^2 + (1 - 3.75)^2 = 9.1875 \rightarrow \sqrt{9.1875} = 3.031$

Weight cuối cùng:

$$w_{BA} = \frac{7.6625}{2.944 \times 3.031} \approx 0.86$$

2.5 5. Ý nghĩa của weight cao

Bảng 2.2: Phản ứng của người dùng đối với Item A và B

User	A	B
U2	5	4
U3	4	5
U4	1	1

Phản ứng gần giống nhau $\Rightarrow$ Correlation cao $\Rightarrow$ Hai sách tương đồng. Ý nghĩa cốt lõi: “Các item có cùng phản ứng với cùng người dùng sẽ có độ tương đồng cao.”

2.6 6. Tính weight giữa B và C

Tương tự ta sẽ thu được:

$$w_{BC} \approx -0.95$$

Điều này hợp lý vì người thích truyện trinh thám thường không thích Doraemon $\Rightarrow$ tương quan âm mạnh.

2.7 7. Predict rating cho U1 với item B

Công thức:

$$s(u, i) = \bar{r}_i + \frac{\sum_{j \in N(i)} w_{ij}(r_{uj} - \bar{r}_j)}{\sum_{j \in N(i)} |w_{ij}|}$$

Thành phần mean-centered:

- Với A: $r_{U1,A} - \bar{r}_A = 5 - 3.75 = 1.25 \Rightarrow$ U1 thích A hơn mức trung bình.
- Với C: $r_{U1,C} - \bar{r}_C = 1 - 2.25 = -1.25 \Rightarrow$ U1 ghét C hơn mức trung bình.

Thay vào công thức:

$$s(U1, B) = 3.33 + \frac{0.86(1.25) + (-0.95)(-1.25)}{|0.86| + |-0.95|} = 3.33 + \frac{2.2625}{1.81} = 3.33 + 1.25$$

2.8 8. Rating dự đoán cuối cùng

$$s(U1, B) = 4.58$$

2.9 9. Giải thích trực quan

Hệ thống suy luận: U1 rất thích sách A, mà A rất giống B. U1 rất ghét C, mà C rất ngược với B. Khả năng rất cao U1 sẽ thích B. Predict $4.58 \approx 5$ sao.

2.10 10. Ý nghĩa của mean-centering

Phần $r_{uj} - \bar{r}_j$ giúp biết User thích item này hơn mức trung bình hay không và loại bỏ bias của item.

2.11 11. Tổng kết thuật toán Item-Item CF

1. Tính similarity giữa các item.
2. Với target user và target item: Lấy các item user đã rate làm neighbors.
3. Chọn top-k neighbors mạnh nhất (thực tế thường khoảng 20).
4. Dùng weighted average để predict rating.
5. Recommend các item có predicted rating cao nhất.

Chương 3

Content-based Filtering

3.1 1. Giới thiệu

Đặc trưng:

- Không có sự tương tác giữa các người dùng, sử dụng feedback history.
- Gợi ý các sản phẩm tương tự với các sản phẩm user đã tương tác trong quá khứ.
- Độ tương đồng ở đây sẽ được định nghĩa dựa trên nội dung của sản phẩm.

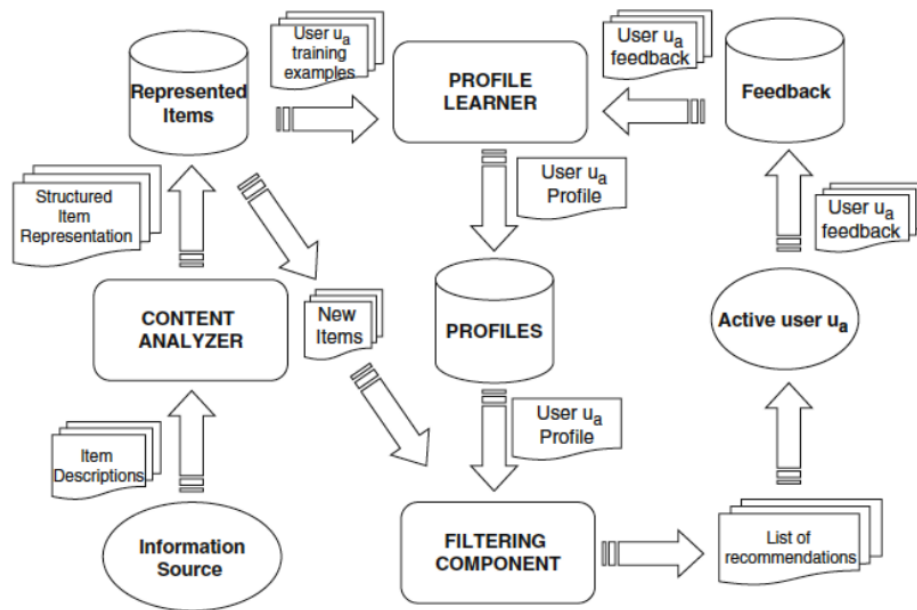
$$s(u, i) = \overline{r_i} + \frac{\sum_{j \in N(i)} w_{ij} (r_{uj} - \overline{r_j})}{\sum_{j \in N(i)} |w_{ij}|}$$

Hình 3.1: Giới thiệu cơ bản về Content-based Filtering

3.2 2. Thuật ngữ

- **Item content:** Sự mô tả một item về mặt đặc trưng của nó.
- **Hồ sơ người dùng (User profile):** Bản mô tả các sở thích của người dùng dựa trên item content đã xuất hiện trong lịch sử phản hồi của họ.
- **Content-based Recommender:** Ước tính xem user profile này match với item content bao nhiêu.

3.3 3. Workflow



Hình 3.2: Sơ đồ Workflow của hệ thống Content-based Filtering

1. Giai đoạn Phân tích Nội dung (Bên trái sơ đồ)

Đây là bước biến các thông tin thô thành dữ liệu mà máy tính có thể hiểu được thông qua Bộ phân tích (Content Analyzer).

2. Giai đoạn Học hồ sơ người dùng (Phía trên & Giữa)

Hệ thống kết hợp giữa nội dung sản phẩm đã tương tác và phản hồi của người dùng để tạo ra tập dữ liệu huấn luyện (Profile Learner).

3. Giai đoạn Lọc và Gợi ý (Phía dưới & Bên phải)

Đây là lúc hệ thống thực sự đưa ra đề xuất cho người dùng thông qua bộ lọc (Filtering Component).

3.4 4. Item Content

Nó có thể là bất cứ dạng gì (Có cấu trúc, hoặc phi cấu trúc).

	Item 1 (Phim A)	Item 2 (Phim B)	Số lượng ratings của User
User 1	5		1
User 2	4	2	2
User 3		5	1
User 4	2		1
User 5		4	1
Số lượng ratings của Item	3	3	

Hình 3.3: Các dạng mô tả của Item Content

3.5 5. Cách biểu diễn vector các mặt hàng

Dễ tính độ tương đồng dựa trên các công thức đã biết và máy tính dễ xử lí.

3.6 6. Mô hình không gian vector (Vector Space Model)

Document d được biểu diễn bằng vector n chiều:

$$\langle w_{1d}, w_{2d}, \dots, w_{nd} \rangle$$

Mỗi phần tử là trọng số của term t trong document d . Có 3 cách để chuyển: Bag of word, Count frequencies, Weigh frequencies.

3.7 7. Bag of word

Đặc điểm là sẽ không quan tâm thứ tự của câu trong document → Lấy được nghĩa tổng quát nhưng không hiểu ngữ cảnh.

Title	Genre	Author	Type	Price	Keywords
The Night of the Gun	Memoir	David Carr	Paperback	29.90	Press and journalism, drug addiction, personal memoirs, New York
The Lace Reader	Fiction, Mystery	Brunonia Barry	Hardcover	49.90	American contemporary fiction, detective, historical
Into the Fire	Romance, Suspense	Suzanne Brockmann	Hardcover	45.90	American fiction, murder, neo-Nazism

Hình 3.4: Mô hình Bag of word

3.8 8. Count frequencies

- **document d** is represented by a n -dimensional **vector**

$$\langle w_{1d}, w_{2d}, \dots, w_{nd} \rangle$$

Hình 3.5: Mô hình Count frequencies

3.9 9. Weigh frequencies

Tần suất xuất hiện của một từ có phản ánh tầm quan trọng?

- 1 từ xuất hiện 10 lần thì mức độ liên quan vẫn hơn 1 lần, nhưng không phải gấp 10 lần.
- Độ quan trọng phụ thuộc vào tần suất xuất hiện từ đó trên tất cả document.

Giải pháp TF-IDF:

$$w_{t,d} = tf_{t,d} \times idf_t$$

3.10 10. Term Frequencies

$$tf_{t,d} = \begin{cases} \log_{10}(1 + f_{t,d}), & \text{if } f_{t,d} > 0 \\ 0, & \text{otherwise} \end{cases}$$

Hàm này cho ta thấy tần suất xuất hiện của một từ không có làm tăng sự đóng góp của nó theo tuyến tính mà là theo hàm log.

3.11 11. Inverse document frequency

$$idf_t = \log_{10} \left(\frac{N}{df_t} \right)$$

Các từ hiếm xuất hiện trên toàn bộ document sẽ liên quan hơn các stopwords. Trong đó N là số lượng document, df_t là số document mà term đó xuất hiện.

$$tf_{t,d} = \begin{cases} \log_{10}(1 + f_{t,d}), & \text{if } f_{t,d} > 0 \\ 0, & \text{otherwise} \end{cases}$$

- examples for raw frequency $\rightarrow$ term frequency:
 - $0 \rightarrow 0$, $1 \rightarrow 0.3$, $10 \rightarrow \sim 1$, $1000 \rightarrow \sim 3$, etc.

Hình 3.6: Minh họa Inverse document frequency (IDF)

3.12 12. Example of after tf-idf

$$idf_t = \log_{10} \left(\frac{N}{df_t} \right)$$

Hình 3.7: Ví dụ minh họa sau khi áp dụng tính toán TF-IDF

3.13 13. User Profile + Matching

Theo phương pháp của Rocchio thì User Profile được định nghĩa như sau: "Một vector đơn nhất là tổ hợp tuyến tính của các vector từ những mục đã được đánh giá trong quá khứ."

Công thức tổng quát của Rocchio:

$$u = \alpha \cdot u_0 + \beta \cdot \frac{1}{|D^+|} \sum_{d^+ \in D^+} d^+ - \gamma \cdot \frac{1}{|D^-|} \sum_{d^- \in D^-} d^-$$

- we can take the **average rating** of the target user...
 - (in U-U CF, we took the average rating of the target item)

$$s(u, i) = \frac{\sum_{j \in I_u} r_{uj}}{|I_u|}$$

Hình 3.8: Công thức Rocchio trong User Profile Matching

3.14 14. Relevant feedback

Bảng 3.1: Các phương pháp xác định vector cơ sở u_0 tùy ngữ cảnh hệ thống

Ngữ cảnh hệ thống	u_0 được tính bằng gì?
User đã dùng lâu năm	Vector Profile cũ lưu trong Database từ phiên trước. $u_0 = u^{(t-1)}$
User mới tinh (Cold Start)	Trung bình cộng các thẻ Tag/Thẻ loại user tick chọn khi đăng ký. $u_0 = \frac{1}{ T^+ } \sum_{v_{tag} \in T^+} v_{tag}$
Hệ thống chạy dựa trên Search	Vector Embedding / TF-IDF của từ khóa người dùng vừa gõ. $u_0 = [\text{tf-idf}(t_1, q), \dots, \text{tf-idf}(t_n, q)]$
Không có bất kỳ dữ liệu nào	Vector toàn số 0 $[0, 0, \dots, 0]$ - hệ thống sẽ dò dẫm gu của user từ đầu.

3.15 15. Ví dụ: Cập nhật Profile người dùng theo sách

Giả sử hệ thống biểu diễn sách trong không gian vector 2 chiều ($n = 2$):

- Trục hoành (x): Mức độ **Trình thám** từ 0 đến 10.
- Trục tung (y): Mức độ **Hài hước / Nhẹ nhàng** từ 0 đến 10.

Bước 1: Xác định vector cơ sở ban đầu

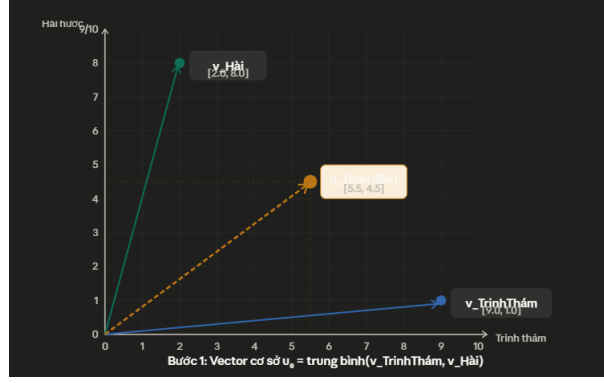
Một người dùng mới đăng ký và chọn 2 thẻ loại yêu thích trong quá trình onboarding:

$$v_{\text{TrìnhThám}} = [9.0, 1.0]$$

$$v_{\text{Hài}} = [2.0, 8.0]$$

Vector cơ sở ban đầu là trung bình cộng của hai vector thẻ loại này:

$$u_0 = \frac{[9.0, 1.0] + [2.0, 8.0]}{2} = [5.5, 4.5]$$



Hình 3.9: Biểu diễn vector sở thích ban đầu u_0 trên không gian 2 chiều

Ý nghĩa: $u_0 = [5.5, 4.5]$ thể hiện sở thích ban đầu của người dùng đang nằm ở mức trung hòa giữa **Trình thám** và **Hải hước**.

Bước 2: Thu thập phản hồi ngược

Sau một thời gian, người dùng đã đọc và phản hồi một số cuốn sách.

Tập tích cực D^+ , tức các sách người dùng đã mua hoặc thích:

$$d_1 = [9.5, 0.5] \quad (\text{Sherlock Holmes})$$

$$d_2 = [8.5, 1.5] \quad (\text{Án mạng trên chuyến tàu})$$

Centroid của tập tích cực là:

$$\text{Centroid}(D^+) = \frac{[9.5, 0.5] + [8.5, 1.5]}{2} = [9.0, 1.0]$$

Tập tiêu cực D^- , tức các sách người dùng bỏ qua hoặc đánh giá thấp:

$$d_3 = [1.0, 9.0] \quad (\text{Doraemon})$$

Do chỉ có một sách trong tập tiêu cực nên:

$$\text{Centroid}(D^-) = [1.0, 9.0]$$

Bước 3: Áp dụng công thức Rocchio

Giả sử các tham số của Rocchio là:

$$\alpha = 1.0, \quad \beta = 1.0, \quad \gamma = 0.5$$

Công thức cập nhật profile người dùng:

$$u = \alpha \cdot u_0 + \beta \cdot \text{Centroid}(D^+) - \gamma \cdot \text{Centroid}(D^-)$$

Thay số vào công thức:

$$u = 1.0 \cdot [5.5, 4.5] + 1.0 \cdot [9.0, 1.0] - 0.5 \cdot [1.0, 9.0]$$

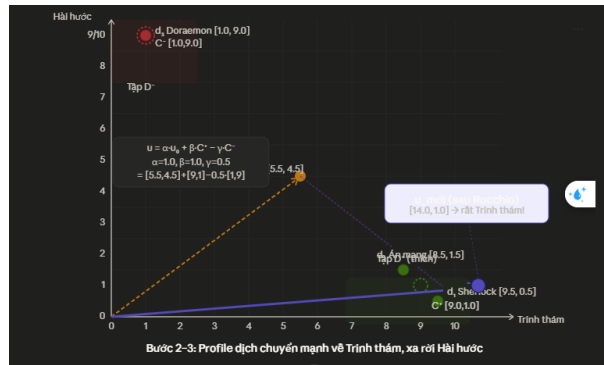
Tính từng thành phần của vector:

$$u_x = 5.5 + 9.0 - 0.5 = 14.0$$

$$u_y = 4.5 + 1.0 - 4.5 = 1.0$$

Kết quả profile mới của người dùng là:

$$u = [14.0, 1.0]$$



Hình 3.10: Sự dịch chuyển của vector sở thích u sau khi cập nhật bằng công thức Rocchio

Phân tích: Profile người dùng đã dịch chuyển rất mạnh về phía **Trình thám**. Thành phần x tăng từ 5.5 lên 14.0, trong khi thành phần y giảm từ 4.5 xuống 1.0. Điều này cho thấy hệ thống đã học được rằng hành vi thực tế của người dùng nghiêng nhiều hơn về sách trình thám, dù ban đầu người dùng cũng chọn thể loại hài hước.

Bước 4: Matching để đưa ra gợi ý

Hệ thống cần lựa chọn giữa 2 cuốn sách mới:

$$d_X = [9.5, 0.5] \quad (\text{Sách X: Trình thám nặng})$$

$$d_Y = [1.0, 9.0] \quad (\text{Sách Y: Truyện cười})$$

Ta sử dụng Cosine Similarity giữa profile người dùng $u = [14.0, 1.0]$ và từng vector sách.

Độ tương đồng với sách X

$$\text{Sim}(u, d_X) = \frac{(14.0 \times 9.5) + (1.0 \times 0.5)}{\sqrt{14.0^2 + 1.0^2} \times \sqrt{9.5^2 + 0.5^2}}$$

$$\text{Sim}(u, d_X) = \frac{133.5}{14.03 \times 9.51} \approx 0.999$$

Độ tương đồng với sách Y

$$\text{Sim}(u, d_Y) = \frac{(14.0 \times 1.0) + (1.0 \times 9.0)}{\sqrt{14.0^2 + 1.0^2} \times \sqrt{1.0^2 + 9.0^2}}$$

$$\text{Sim}(u, d_Y) = \frac{23.0}{14.03 \times 9.05} \approx 0.181$$



Hình 3.11: So sánh độ tương đồng cosine giữa profile người dùng u với Sách X và Sách Y

Kết luận

Vì:

$$\text{Sim}(u, d_X) = 0.999 > \text{Sim}(u, d_Y) = 0.181$$

nên hệ thống sẽ ưu tiên gợi ý **Sách X** cho người dùng.

Ví dụ này cho thấy công thức Rocchio có thể tự động “uốn nắn” hồ sơ người dùng dựa trên phản hồi thực tế. Ban đầu người dùng có vẻ thích cả trình thám và hài hước, nhưng sau khi quan sát hành vi đọc sách, hệ thống nhận ra người dùng nghiêng mạnh về sách trình thám hơn.

Chương 4

Hệ thống Hybrid Recommender đơn giản

Sau khi đã tìm hiểu hai hướng chính là **Collaborative Filtering** và **Content-based Filtering**, ta có thể kết hợp cả hai để tạo thành một hệ thống gợi ý lai, hay còn gọi là **Hybrid Recommender System**.

Ý tưởng chính của hệ thống hybrid là: thay vì chỉ dựa vào một nguồn thông tin duy nhất, hệ thống sẽ kết hợp nhiều điểm số khác nhau để đưa ra kết quả gợi ý cuối cùng.

Trong báo cáo này, ta xét một dạng hybrid đơn giản nhất:

$$\text{Score}_{\text{hybrid}}(u, i) = \lambda \cdot \text{Score}_{CF}(u, i) + (1 - \lambda) \cdot \text{Score}_{CB}(u, i)$$

Trong đó:

- $\text{Score}_{\text{hybrid}}(u, i)$ là điểm gợi ý cuối cùng của item i đối với user u .
- $\text{Score}_{CF}(u, i)$ là điểm dự đoán từ Collaborative Filtering.
- $\text{Score}_{CB}(u, i)$ là điểm dự đoán từ Content-based Filtering.
- λ là hệ số điều chỉnh mức độ tin tưởng vào Collaborative Filtering.
- $(1 - \lambda)$ là hệ số điều chỉnh mức độ tin tưởng vào Content-based Filtering.

Nếu λ càng lớn, hệ thống càng ưu tiên kết quả từ Collaborative Filtering. Ngược lại, nếu λ càng nhỏ, hệ thống càng ưu tiên kết quả từ Content-based Filtering.

4.1 1. Tại sao cần Hybrid Recommender?

Collaborative Filtering và Content-based Filtering đều có điểm mạnh và điểm yếu riêng.

Collaborative Filtering có thể khai thác hành vi của nhiều người dùng để tìm ra các mẫu tương tác ẩn. Ví dụ, nếu nhiều người thích sách A cũng thích sách B, hệ thống có thể gợi ý sách B cho người đã thích A. Tuy nhiên, phương pháp này gặp khó khăn khi user mới hoặc item mới chưa có nhiều tương tác.

Content-based Filtering có thể gợi ý dựa trên nội dung của item, chẳng hạn như title, author, category, description. Phương pháp này xử lý tốt hơn trong trường hợp item mới

vì chỉ cần có mô tả nội dung là có thể tính độ tương đồng. Tuy nhiên, nó dễ bị giới hạn trong những item giống với lịch sử cũ của người dùng, làm giảm sự đa dạng trong gợi ý.

Do đó, hệ thống hybrid giúp tận dụng ưu điểm của cả hai hướng:

- Dùng Collaborative Filtering để học từ hành vi cộng đồng.
- Dùng Content-based Filtering để khai thác nội dung sản phẩm.
- Giảm ảnh hưởng của cold-start.
- Tăng độ ổn định và độ chính xác của hệ thống gợi ý.

4.2 2. Công thức cộng trọng số đơn giản

Giả sử hệ thống cần dự đoán mức độ phù hợp giữa user u và item i .

Ta có hai điểm số:

$$s_{CF}(u, i)$$

là điểm dự đoán từ Item-Item Collaborative Filtering, và:

$$s_{CB}(u, i)$$

là điểm dự đoán từ Content-based Filtering.

Khi đó, điểm hybrid được tính bằng:

$$s_{hybrid}(u, i) = \lambda s_{CF}(u, i) + (1 - \lambda) s_{CB}(u, i)$$

Trong đó:

$$0 \leq \lambda \leq 1$$

Một số trường hợp đặc biệt:

- Nếu $\lambda = 1$, hệ thống chỉ dùng Collaborative Filtering:

$$s_{hybrid}(u, i) = s_{CF}(u, i)$$

- Nếu $\lambda = 0$, hệ thống chỉ dùng Content-based Filtering:

$$s_{hybrid}(u, i) = s_{CB}(u, i)$$

- Nếu $\lambda = 0.5$, hệ thống xem hai mô hình quan trọng như nhau:

$$s_{hybrid}(u, i) = 0.5 s_{CF}(u, i) + 0.5 s_{CB}(u, i)$$

4.3 3. Lưu ý về chuẩn hóa điểm số

Trước khi cộng hai điểm số lại với nhau, ta cần đảm bảo chúng nằm trên cùng một thang đo.

Ví dụ:

- Collaborative Filtering có thể trả về rating dự đoán trong khoảng từ 1 đến 5.
- Content-based Filtering dùng cosine similarity, thường nằm trong khoảng từ 0 đến 1.

Nếu cộng trực tiếp hai điểm này, điểm CF sẽ có ảnh hưởng lớn hơn chỉ vì thang đo của nó lớn hơn. Do đó, cần chuẩn hóa rating của CF về khoảng $[0, 1]$.

Với rating nằm trong khoảng từ 1 đến 5, ta có thể chuẩn hóa như sau:

$$s_{CF}^{norm}(u, i) = \frac{s_{CF}(u, i) - 1}{5 - 1} = \frac{s_{CF}(u, i) - 1}{4}$$

Sau khi chuẩn hóa, công thức hybrid trở thành:

$$s_{hybrid}(u, i) = \lambda s_{CF}^{norm}(u, i) + (1 - \lambda) s_{CB}(u, i)$$

Khi đó cả hai điểm số đều nằm trên cùng một thang đo $[0, 1]$, giúp việc cộng trọng số công bằng hơn.

4.4 4. Ví dụ cụ thể

Giả sử hệ thống cần gợi ý sách cho user $U1$. Ta xét hai cuốn sách mới:

- Sách X: Trinh thám nặng.
- Sách Y: Truyện cười nhẹ nhàng.

Từ mô hình Collaborative Filtering, hệ thống dự đoán:

$$s_{CF}(U1, X) = 4.6$$

$$s_{CF}(U1, Y) = 2.0$$

Vì điểm CF đang nằm trên thang rating từ 1 đến 5, ta chuẩn hóa về $[0, 1]$:

$$s_{CF}^{norm}(U1, X) = \frac{4.6 - 1}{4} = 0.9$$

$$s_{CF}^{norm}(U1, Y) = \frac{2.0 - 1}{4} = 0.25$$

Từ mô hình Content-based Filtering, hệ thống tính được cosine similarity:

$$s_{CB}(U1, X) = 0.999$$

$$s_{CB}(U1, Y) = 0.181$$

Giả sử ta chọn:

$$\lambda = 0.6$$

Điều này có nghĩa là hệ thống tin vào Collaborative Filtering nhiều hơn một chút so với Content-based Filtering.

Tính điểm hybrid cho sách X

$$s_{hybrid}(U1, X) = 0.6 \cdot 0.9 + 0.4 \cdot 0.999$$

$$s_{hybrid}(U1, X) = 0.54 + 0.3996 = 0.9396$$

Tính điểm hybrid cho sách Y

$$s_{hybrid}(U1, Y) = 0.6 \cdot 0.25 + 0.4 \cdot 0.181$$

$$s_{hybrid}(U1, Y) = 0.15 + 0.0724 = 0.2224$$

Vì:

$$s_{hybrid}(U1, X) = 0.9396 > s_{hybrid}(U1, Y) = 0.2224$$

nên hệ thống sẽ ưu tiên gợi ý **Sách X** cho user $U1$.

4.5 5. Quy trình thuật toán Hybrid đơn giản

Quy trình xây dựng hệ thống hybrid cộng trọng số có thể được mô tả như sau:

1. Với mỗi user u và item ứng viên i , tính điểm Collaborative Filtering:

$$s_{CF}(u, i)$$

2. Chuẩn hóa điểm CF về khoảng $[0, 1]$ nếu cần:

$$s_{CF}^{norm}(u, i) = \frac{s_{CF}(u, i) - r_{min}}{r_{max} - r_{min}}$$

3. Tính điểm Content-based Filtering:

$$s_{CB}(u, i) = \cos(u, d_i)$$

4. Kết hợp hai điểm số bằng công thức cộng trọng số:

$$s_{hybrid}(u, i) = \lambda s_{CF}^{norm}(u, i) + (1 - \lambda) s_{CB}(u, i)$$

5. Sắp xếp các item theo $s_{hybrid}(u, i)$ giảm dần.
6. Gợi ý Top- k item có điểm cao nhất cho người dùng.

4.6 6. Cách chọn hệ số λ

Hệ số λ quyết định mức độ ưu tiên giữa hai mô hình.

- Nếu hệ thống có nhiều dữ liệu rating và hành vi người dùng, có thể chọn λ lớn, ví dụ:

$$\lambda = 0.7$$

Khi đó hệ thống tin vào Collaborative Filtering nhiều hơn.

- Nếu hệ thống có ít rating nhưng item có mô tả nội dung tốt, có thể chọn λ nhỏ hơn, ví dụ:

$$\lambda = 0.3$$

Khi đó hệ thống tin vào Content-based Filtering nhiều hơn.

- Nếu chưa biết mô hình nào tốt hơn, có thể bắt đầu với:

$$\lambda = 0.5$$

rồi điều chỉnh dựa trên kết quả thực nghiệm.

Trong thực tế, λ có thể được chọn bằng cách thử nhiều giá trị khác nhau trên tập validation, ví dụ:

$$\lambda \in \{0.1, 0.2, 0.3, \dots, 0.9\}$$

Sau đó chọn giá trị cho kết quả tốt nhất theo metric đánh giá như RMSE, MAE, Precision@K, Recall@K hoặc NDCG@K.

4.7 7. Ưu điểm và hạn chế

Ưu điểm

- Kết hợp được thông tin từ hành vi người dùng và nội dung item.
- Giảm phụ thuộc vào một mô hình duy nhất.
- Giảm vấn đề cold-start so với Collaborative Filtering thuần túy.
- Dễ cài đặt và dễ giải thích.
- Có thể điều chỉnh linh hoạt bằng hệ số λ .

Hạn chế

- Cần chuẩn hóa điểm số trước khi cộng.
- Việc chọn λ ảnh hưởng lớn đến chất lượng gợi ý.
- Nếu cả hai mô hình con đều cho kết quả kém thì mô hình hybrid cũng khó đạt kết quả tốt.
- Công thức cộng trọng số tuyến tính khá đơn giản, chưa học được quan hệ phức tạp giữa các đặc trưng.

4.8 8. Tổng kết

Hệ thống Hybrid Recommender đơn giản có thể được hiểu là cách kết hợp điểm số từ Collaborative Filtering và Content-based Filtering bằng một công thức cộng trọng số:

$$s_{hybrid}(u, i) = \lambda s_{CF}^{norm}(u, i) + (1 - \lambda) s_{CB}(u, i)$$

Cách tiếp cận này đơn giản nhưng hiệu quả trong nhiều trường hợp vì nó tận dụng được cả hai nguồn thông tin:

- Collaborative Filtering học từ hành vi của cộng đồng người dùng.
- Content-based Filtering học từ nội dung và đặc trưng của item.

Cuối cùng, hệ thống sẽ gợi ý những item có điểm $s_{hybrid}(u, i)$ cao nhất cho người dùng.